Listing 3.12 Rendering experiment.html after your changes in the Django REPL

```
>>> from django.template import Engine, Context
>>> engine = Engine.get_default()
>>> t = engine.get_template("experiment.html")
>>> data = {"instrument":"flute"}
>>> t.render(Context(data))
"I love the sound a flute makes.\n
➥ A shiny flute is good.\n\n\n
➥ Don't put a flute too close to your ear.\n\n"
>>> data = {"instrument":"<b>sax</b>"}
>>> t.render(Context(data))
"I love the sound a &lt;b&gt;sax&lt;/b&gt; makes.\n
➥ A shiny <b>sax</b> is good.\n\n\n
➥ Don't put a <b>sax</b> too close to your ear.\n\n"
```

Without modification, the `<b>` tags get escaped. Using either the `| safe` filter or the `{% autoescape %}` tag allows you to include HTML inside your data.

There is one more way to affect auto-escaping behavior: change the data. Django has a subclass of a Python `str` called `SafeString`. You get a `SafeString` by calling the `mark_safe()` utility method on a string. Alternatively, If you are dynamically building snippets of HTML, you can use `format_html()`, which also returns a `SafeString`. Any string marked as safe will not be escaped. Operations on `SafeString` objects return regular `str` objects; if you need to modify a `SafeString`, you will have to mark it as safe again.

Let's play with these utilities in the same REPL session. Use the same template as before, but this time, modify the context data to contain a string marked as safe. The `mark_safe()` function takes the string you want to mark safe as an argument. Add a safe string with a `<br/>` tag to the `instruments` context, and then render it, as in the following listing.

Listing 3.13 Using `mark_safe()` to mark data as safe in the Django REPL

```
>>> from django.utils.safestring import mark_safe
>>> instrument = mark_safe("drum<br/>")
>>> data = {"instrument":instrument}
>>> t.render(Context(data))
"I love the sound a drum<br/> makes.\n
➥ A shiny drum<br/> is good.\n\n\n
➥ Don't put a drum<br/> too close to your ear.\n\n"
```

The `format_html()` function works like `str.format()`, using braces as placeholders and passing in arguments used to populate the string. The `format_html()` method can save a lot of typing if you're building a chunk of HTML, where you'd otherwise need to compose and use `mark_safe()` on individual pieces. In the same REPL session, build a phrase that contains paired `<i>` tags with a placeholder between them, like in the following listing.